



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۱۲ آذر

فصل چهارم: لایه *Network* بخش *Data – Plane* اسلاید ۶۲ تا ۸۰ [۱]

خب نمی‌توانیم که تمام روترها را در یک روز آپگرید کنیم و از *IPv4* به *IPv6* آپگرید کنیم. پس هیچ روزی موسوم به روز *flag day* نداریم که همزمان همه را ارتقا دهیم. اما این ارتقا را میتوانیم به صورت پله پله انجام بدهیم. اینگونه ممکن است همزمان یک *mix* از روترهای *IPv4* و *IPv6* در شبکه وجود داشته باشد. پس از آنجایی که این روترها هر کدام هدر مخصوص به تکنولوژی خود را می‌فهمند، نیاز به روشی داریم که بتوان پیام را از میان مخلوطی از روترهای *IPv4* و *IPv6* منتقل کرد.

روش *Tunneling*

در این روش، در صورتی که یک بسته دیتاگرام *IPv6* به روتری رسید روتر بعدی آن که تنها تکنولوژی *IPv4* را پشتیبانی میکند، آن بسته را به عنوان *payload* در یک بسته دیتاگرام *IPv4* قرار می‌دهیم و به آن روتر می‌دهیم. حال *source* و *destination* در هدر *IPv4* را برابر با دو روتری مرزی قرار می‌دهیم که بین آن‌ها مسیر *IPv4* قرار دارد و تمام روترهای آن مسیر از *IPv4* هستند. توجه کنید در خود هدرهای *IPv6* هدرهای *source* و *destination* داریم که باید همان مبدا و مقصد اولیه باشند. اما برای مسیرهای *IPv4*، در سر مسیر را که روترهای مرزی هستند و هر دو تکنولوژی *IPv4* و *IPv6* ساپورت میکنند قرار می‌دهیم و *source* و *destination* هدر بسته *IPv4* را این دو روتر مرزی قرار می‌دهیم و نه روترهای مبدا و مقصد. بسته *IPv6* هم در قالب *payload* در بسته *IPv4*، *encapsulate* شده است. یک فیلد *flow* هم در *IPv6* است که در هدر *IPv4* نیست. برای بسته‌های مرتب با هم.

Generalized forwarding

در روش‌های مبتنی بر نرم افزار برای مسیر یابی *SDN*، *data plane* و *control plane* از هم جدا شدند، مثل روش سنتی نیست که همه روترها توابع پیچیده مسیر یابی را در خود داشته باشند، در عوض مسیریابی توسط یک سیستم کنترلر بیرونی و منطقاً متمرکز انجام میشود. *remote controller* این کنترلر در هر لحظه گراف شبکه را دارد.

طبق آن مسیر یابی میکند و اطلاعات لازم موسوم به *flow table* را که همان *forwarding table* تعمیم یافته و *generalized* است، به تابع *control agent* در هر روتر میدهد (این تابع ساده، راه ارتباطی میان کنترلر و روتر است) نکته مهم این *flow table* است که نسبت به حالت قبلی گفته شده در *forwarding table* که تنها میتواندست عمل *forwarding* را انجام دهد، اینجا کارهای متنوع بیشتری صورت میگیرد که به تفصیل به آن میپردازیم. یعنی بعد از *match* کردن اولیه، *action* های مختلفی علاوه بر فورواردینگ میتواند صورت بگیرد.

OpenFlow

یک نوع پیاده سازی از همان مفهوم *generalized forwarding* است. این روش ۴ عمل اصلی را انجام میدهد:

۱. *match*: اینکه از روی هدرها و *pattern* های مربوطه و یک سری قوانین مربوط به هدرهای مختلف داده، آن ها را تطبیق دهد.
۲. *action*: اینکه بعد از تطبیق، بسته را *forward* کند یا *drop* کند یا *modify* کند و یا به کنترلر انتقالش دهد.
۳. *priority*: اینکه بین بسته های با یک الگوی هدر یکسان و *overlapping*، اولویت بگذارد.
۴. *counters*: اینکه یک سری اطلاعات آماری از بسته ها جمع آوری کند، مثلا تعداد بایت ها یا بسته های عبوری و غیره

این الگوها طبق یک سری *rule* انجام میشود. مثلا اگر *match* بر اساس آدرس *MAC* باشد، به عنوان *switch* در نظر گرفته میشود. *layer 2 destination – based forwarding*. اینجا احتمالا پیام را به نوعی برودکست میکند که به آن *flood* میگویند

اگر طبق آدرس و پورت مقصد باشد (یا به علاوه آدرس و پورت مبدا)، به عنوان روتر.

اگر در *action*، آن پورت و آدرس مبدا را به پورت و آدرس مبدا دیگری مپ کند و *rewrite* کند، به عنوان *NAT*، و اگر هم جلوی یک سری پکت ها از پورت و آدرس خاص را بگیرد، به عنوان *firewall* در نظر گرفته می شود. که وظیفه *deny* یا *permit* را دارد.

middlebox

یک باکس میانی در مسیر داده بین مبدا و مقصد، که میتواند یک تابع متفاوت را اجرا کند غیر از توابع روال و استاندارد. قابلیت برنامه ریزی دارد (مثلا عمل های محلی در سطح سوئیچ)

شبکه های نرم افزار محور، این توانایی را میدهد که مدیریت پیکربندی کارآمدی که در مفهومی موسوم به *NFV* = *network function virtualization* امکان پذیر شده است، را داشته باشیم، چه در سطح *cloud*، چه خصوصی

چه عمومی.

میتوانیم جعبه‌های فایروال، لود بالانسر، NAT و غیره را میتوان در NFV با قابلیت برنامه ریزی شبکه، محاسبات و ذخیره سازی، پیاده سازی کرد.

whitebox

این نوع سوویچ‌ها، توسط SDN مدیریت میشوند و ISP ها میتوانند OS مربوطه و لازم را روی آن دانلود کند و قرار دهند. پس مبتنی بر پردازنده‌های همه منظوره هستند و همه منظوره هستند. و ارزانتر. پس ISP ها سوویچ‌های *whitebox* و SDN را در دیتاسنتر و NFV را در هسته به کار میگیرند و مقیاس پذیری را با صرف هزینه کم انجام میدهند.

از انواع معروف *middle box* ها:

- NAT در خانه و موسسات و شبکه‌های سلولی
- *firewall* ها و *IDS* یا *intrusion detection system* ها یا سیستم‌های تشخیص نفوذ در شرکت ها و موسسات و ISP ها
- *load balancer* در شبکه‌های سیار *mobile net* و ISP ها و شرکت ها و مراکز داده.
- *cache* ها در *CDN* ها و ISP ها و شبکه‌های سیار *mobile*
- *application – specific* در ISP ها و *CDN* ها

مدل ساعت شنی

همینطور که قبلاً هزاربار گفتیم، لایه شبکه با یک پروتکل $IP = Internet\ protocol$ کار میکند و لایه‌های دورتر با پروتکل‌های بیشتر (لایه‌های ۱ و ۵ پروتکل‌های متنوع تری از لایه‌های ۲ و ۴ دارند) این سادگی برای *connectivity* است و اینکه بخواهند توسط میلیارد ها دیوایس پیاده سازی شود و پیچیدگی را سمت *edge* آوردند.

البته به دلیل اضافه شدن *middle box* ها یک سری پروتکل مربوط به NFV و *caching* و *load balancing* و *firewall* و غیره هم اضافه شده است به آن. که در *core* شبکه *operate* میکنند.

البته بحث شده که *intelligence* کجای سیستم باشد؟

- بعضی ها قبلا میگفتند درون شبکه باشد. مثلا در شبکه های تلفنی سنتی (در هسته شبکه سویچ های هوشمند و کامپیوتری بود و در لبه همان دستگاه های تلفن بودند که هوشمند نبودند)
- بعدا گفتند که در *edge* شبکه باشد و به صورت *end to end* باشد. دارای *host* های کامپیوتری با پردازنده خوب.
- جدیدا این ایده را تقویت کرده و یک سری محاسبات هم به صورت *fog computing* در *edge* شبکه انجام میدهند. درواقع مفهوم *edge computing* اضافه شد و مراکز بزرگی که محاسبات بسیار بزرگتری نسبت به کامپیوتر های شخصی انجام میدهند در اینجا قرار میگیرند.

فصل پنجم: لایه *Network* بخش *Control – Plane* اسلاید ۱ تا ۱۵ [۲]

برخی از مباحث مطرح شده، تا اسلاید ۹، در اسلاید های فصل ۴ آمده است و برای همین از تکرار آن اجتناب میکنم.

انواع روتینگ

- *unicast* یا تک پخش: یعنی پیام به یک مقصد خاص ارسال میشود
- *broadcast* یا همه پخش: یعنی به همه ارسال میشود.
- *multicast* یا چند پخش: یعنی پیام به همه اعضای یک زیرگروه از شبکه ارسال میشود
- *anycast* یا هر پخش: یعنی پیام به یک عضو(هر عضوی میتواند باشد) از یک زیر گروه شبکه ارسال میشود. ارسال به یک عضو کفایت میکند.

شبکه را به صورت گراف مدل سازی میکنیم. هر روتر را با یک راس و هر لینک را با یک یال نشان میدهیم. برای هر لینک، هزینه لینک را نیز در نظر میگیریم. که معمولا ۱ است اما با توجه به پهنای باند، ازدحام و تاخیر، میتواند ارتباط مستقیم و یا معکوس به آنها داشته باشد و هزینه آن بیشتر از ۱ شود. هزینه یک مسیر برابر مجموع هزینه لینک های آن مسیر است. ما دنبال مسیر با هزینه کمینه بین مبدا و مقصد هستیم و الگوریتم های روتینگ را طبق آن انجام میدهیم.

انواع دسته بندی الگوریتم های روتینگ

- با دانش کامل (*global*):

در این نوع الگوریتم‌ها دانش کامل از *topology* شبکه و هزینه لینک‌ها وجود دارد. این نوع الگوریتم‌ها معروف با الگوریتم‌های *link state* این اطلاعات قبل از آغاز الگوریتم، توسط پروتکل *link state dissemination* در اختیار گره‌ها قرار میگیرد.

• با دانش ناقص: اطلاعات هر گره محدود به اطلاعات همسایه‌های مستقیماً متصل است اول. اما به صورت *iterative* دانش خود را نسبت به همسایه‌ها و همسایه‌های همسایه افزایش میدهد. الگوریتم‌های *distance vector* یا بردار فاصله از این نوع هستند.

دسته بندی الگوریتم‌ها طبق متمرکز یا توزیع شده بودن

• متمرکز: مثل *SDN*

• غیر متمرکز: مثل روش‌های سنتی و الگوریتم‌های *link state* و *distance vector*

دسته بندی الگوریتم‌ها طبق *static* یا *dynamic* بودن

این نوع دسته بندی مربوط به ذات *topology* شبکه است. و نه ناشی از ذات الگوریتم. اگر توپولوژی تغییرات آهسته داشته باشد، *static* و اگر تغییرات زیاد باشد، الگوریتم بکار رفته، *dynamic* و پویا می‌باشد.

سوال: پویایی شبکه چگونه به الگوریتم منعکس میشود؟

۱. روش *proactive*: به صورت تناوبی پیام‌های *preiodic update* را به همسایه‌ها میفرستد. ممکن است بین دو پیام تناوبی متوالی، هیچ تغییری نباشد یا تغییرات متعددی رخ داده باشد و دیر به اطلاع همسایه‌ها برسد.

۲. روش *reactive*: هر گاه تغییرات در یک لینک رخ دارد، روترهای متصل به آن لینک، پیام‌ها را به همسایه‌ها منتقل میکنند.

بررسی پروتکل‌های *link state*

این نوع الگوریتم‌ها طبق الگوریتم مسیریابی دایکسترا می‌باشد. قبل از شروع الگوریتم، با همه پخش و *broadcast* اطلاعات لینک‌ها به همه گره‌ها ارسال میشود. پس هر گره توپولوژی شبکه و هزینه لینک‌ها دارند و همه دانش یکسانی دارند و گراف شبکه را میدانند.

الگوریتم دایکسترا

این الگوریتم، کمینه فاصله مبدا تا تک تک گره‌ها را محاسبه میکند. و طبق آن جدول فورواردینگ را میسازد. این الگوریتم به صورت تکراری و *iterative* عمل میکند و تا مرحله k ام کمینه فاصله مبدا تا k گره را میدانیم.

پیاده سازی الگوریتم دایکسترا

هزینه لینک ها را با $c(v, u)$ بین راس های v و u در نظر میگیریم. اگر این دو لینک همسایه مستقیم نباشند، $c(v, u) = \infty$ است.

کمینه فاصله راس v تا مبدا را هم در متغیر $D(v)$ ذخیره میکنیم.

ابتدا فقط راس مبدا u را بررسی میکنیم. اول به ازای تمام همسایه های آن که با v نشان میدهیم، داریم: $D(v) = c(u, v)$ و در غیر اینصورت $D(v) = \infty$

حال از بین راس هایی که بررسی نکردیم (مثلا تا دور اول فقط راس مبدا را بررسی کرده بودیم و در N' قرار داشت ، پس به غیر آن) راس w که کوچک ترین $D(w)$ را دارد را برمیگزینیم.

به ازای تک تک همسایه هایش، که تا حالا بررسی نکردیم (در N' نیستند) این محاسبه را انجام میدهیم: $D(v) = \min(D(v), D(w) + c(w, v))$ بعد از انجام این کار برای همه نود های همسایه دیده خارج از N' ، نود w را در N' اضافه میکنیم.

این کار را انقدر انجام میدهیم تا همه نود ها در N' بیایند و همه بررسی شده باشند.

فصل پنجم: لایه $Network$ بخش $Control - Plane$ اسلاید ۱۶ تا ۲۰ [۳]

خب ابتدا مثالی از الگوریتم دایکسترا حل شد. برای هر نود، وقتی که عمل آپدیت را انجام میدهیم، نود قبلی که با آن به اون نود رسیده را هم در جدول مینویسیم، که اینگونه بتوانیم کوتاه ترین مسیر از مقصد تا مبدا را پیدا کنیم چون هر نود، نود قبلی خود را در مسیر کمینه میداند.

مثلا اگر در ستون $D(z)$ در ردیف آخرش نوشته باشیم y ، یعنی $y = p(z)$ و y نود قبل از z در مسیر کمینه است و فاصله تا z هم در مسیر کمینه ۱۲ است.

اردر الگوریتم دایکسترا، چون در هر $iteration$ باید تمام نود های دیده نشده را به نوعی بررسی کنیم، و این کار را در n تا $iteration$ انجام دهیم، از اردر $n \times (n - 1)/2$ یا همان $O(n^2)$ است.

مشکل $oscillations$ در الگوریتم دایکسترا

چیزی که فهمیدم، احتمالا اینه که اگر مسیر های مختلف تقریبا یک بار ترافیکی داشته باشند، و می خواهیم بدانیم ترافیکمان را به کدام مسیر بفرستیم، هر بار که یک مسیری را انتخاب کنیم، آن مسیر دیگر، بار کمتری خواهد داشت و به آن سوییچ میکنیم و این کار به طور مداوم و پی در پی انجام میشود و هی مسیر را عوض میکنیم.

مراجع

- [1] “ch4-v8-slide-62-to-80,” <https://drive.google.com/drive/folders/>, 2022.
- [2] “ch5-slide-1-to-15,” <https://drive.google.com/drive/folders/>, 2022.
- [3] “ch5-slide-16-to-39,” <https://drive.google.com/drive/folders/>, 2022.